

Camera Pose Estimation Tool

Handoff & User Documentation

Version 1.0 — May 2026

1. Introduction

This tool is an interactive simulator for **camera pose estimation** developed as part of a computer vision project. It estimates the relative pose (rotation $\mathbf{R}$ and translation $\mathbf{t}$) between two cameras from 2D point correspondences, following the methodology used in ORB-SLAM.

What the tool does

From an automatically generated synthetic 3D scene, the tool:

- Projects 3D points onto both camera image planes.
- Estimates the pose of camera 2 relative to camera 1 using two methods:
 - Homography matrix $\mathbf{H}$ — suited for planar scenes (points lying on a common plane).
 - Fundamental matrix $\mathbf{F}$ — suited for general (non-planar) 3D scenes.
- Automatically selects the most reliable method using an ORB-SLAM-inspired score and ratio.
- Displays results in an interactive 3D view and measures rotation/translation errors against ground truth.

2. Prerequisites & Installation

Python version

Python **3.9 or higher** is required. Python 3.10–3.12 is recommended for best compatibility with all dependencies.

Installing dependencies

From the project root directory, run:

```
| pip install -r requirements.txt
```

Launching the interface

From the project root:

```
| python pipeline/gui_tool.py
```

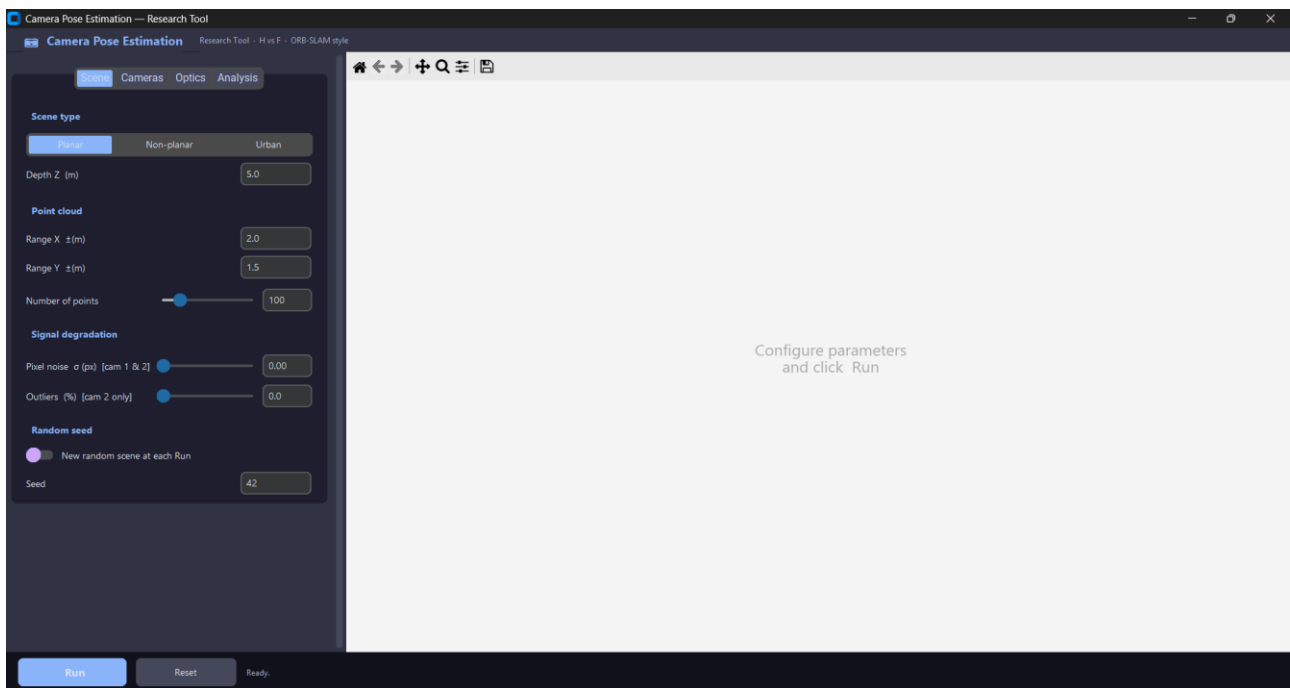
Note: If using Anaconda, ensure the active environment has all packages installed. Python 3.13+ may introduce compatibility issues with some dependencies — using a dedicated virtual environment (`conda create -n pose python=3.11`) is recommended.

3. User Interface Guide

General layout

The interface is divided into two main areas:

- **Left column** — a scrollable sidebar containing 4 parameter tabs and action buttons at the bottom.
- **Right area** — a matplotlib canvas that displays results after each Run. Before the first run, it shows a placeholder: "Configure parameters and click Run".



Tab — Scene



Scene type

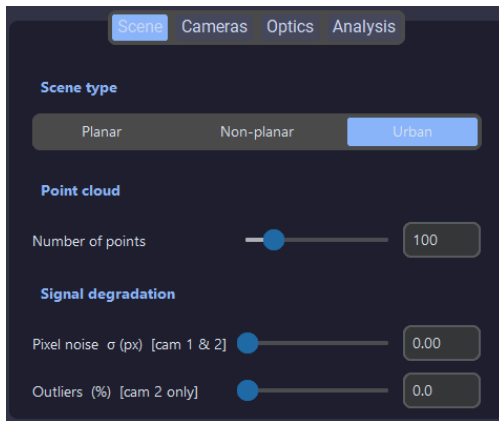
A segmented button at the top lets you choose between three scene types:

Planar — all 3D points are generated at the same depth Z (a single front-parallel plane). This is the ideal case for homography estimation: H is exactly valid for coplanar points.

- **Depth Z (m)** — the single depth at which all points are placed. Typical value: 5.0 m.

Non-planar — 3D points are distributed randomly across a volume. Depths vary between Z min and Z max. This is the ideal case for the fundamental matrix F .

- **Z min (m) / Z max (m)** — depth range of the generated points.



Urban — a fixed synthetic urban environment made of three geometric objects:

- **Building** — rectangular box (3 m × 3 m × 6 m height) placed at Z = 10 m.
- **Tree** — a thin cylindrical trunk topped by a hemispherical dome (mushroom shape), placed at Z = 8 m on the left.
- **Car** — two stacked boxes (body + roof), placed at Z = 7 m on the right.

The geometry is fixed; only the number of points varies.

Point cloud section

These parameters control the spatial distribution of generated points.

Parameter	Applies to	Description
Range X ±(m)	Planar, Non-planar	Horizontal extent of the scene in meters either side of the central axis.
Range Y ±(m)	Planar, Non-planar	Same principle on the vertical axis.
Number of points	All	Total number of 3D points generated. Range: 8 to 500. In Urban mode, points are split across the three objects: 55% building, 25% tree, 20% car.

⚠ In Urban mode, fewer than 100 points makes the shapes hard to recognise visually. Use 300–400 points for a clear view of the scene.

Signal degradation section

These parameters simulate imperfect acquisition conditions.

Parameter	Description
Pixel noise σ (px)	Zero-mean Gaussian noise with standard deviation σ added to the 2D pixel coordinates of both cameras. $\sigma = 0.0$: ideal conditions. $\sigma = 1.0$: realistic precision for a good keypoint detector. $\sigma = 3.0$ – 5.0 : heavy degradation.
Outliers (%) [cam 2]	Percentage of erroneous correspondences injected into camera 2 only. An outlier replaces a projected point's pixel position with a random position anywhere in the image — simulating false keypoint matches. 0%: no outliers. >0% of matches are false (RANSAC should handle these).

Random seed section (Planar and Non-planar only)

- **"New random scene at each Run" switch** — when OFF (default): the same seed is used every Run, so the scene is identical across runs (useful for comparing H and F on exactly the same data). When ON: a new random seed between 0 and 99 999 is drawn at each Run.
- **Seed field** — the numeric seed value (default: 42). Can be edited manually to reproduce a specific scene. When the random switch is ON, this field updates automatically after each Run — useful for noting an interesting scene and reproducing it later.

Tab — Cameras

Scene **Cameras** Optics Analysis

Camera 1 — Reference

Rotation rx ry rz (deg)

Translation tx ty tz (m)

Camera 2

Rotation rx ry rz (deg)

Translation tx ty tz (m)

Frame: $X_{cam} = R @ X_{world} + t$
 $R = R_z \cdot R_y \cdot R_x$ (extrinsic Euler, deg)
Cam 1 identity by default → world reference

This tab defines the pose of both cameras in the world frame. The rotation is built in the order $R_z \cdot R_y \cdot R_x$ (extrinsic Euler angles).

Camera	Parameter	Default	Description
Camera 1 (reference)	rx ry rz (deg)	0 / 0 / 0	Zero rotation: camera 1 sits at the world origin looking in the +Z direction. In practice, Camera 1 is never modified — it defines the reference frame.
Camera 1 (reference)	tx ty tz (m)	0 / 0 / 0	Zero translation: the world origin.
Camera 2	ry (deg)	8°	8° rotation around Y: slight horizontal pan of camera 2 relative to camera 1.
Camera 2	tx (m)	0.4 m	0.4 m offset along X. These default values give a realistic stereo setup.

Tab — Optics

Scene Cameras **Optics** Analysis

Focal lengths

fx (px)

fy (px)

Principal point

cx (px)

cy (px)

Image resolution

Width W (px)

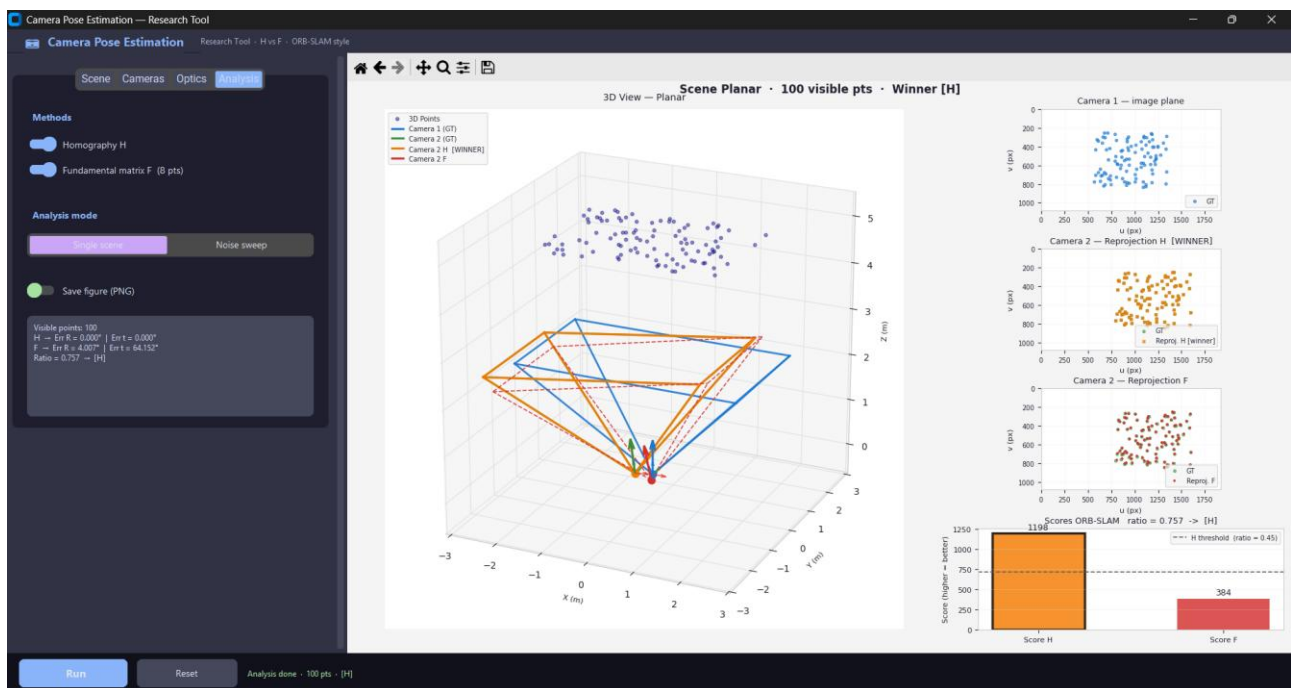
Height H (px)

FOV → 87.7° × 56.7°

This tab defines the intrinsic parameters shared by both cameras.

Parameter	Default	Description
fx (px)	1000	Horizontal focal length in pixels. Defines the horizontal magnification of the camera.
fy (px)	1000	Vertical focal length in pixels.
cx (px)	960	Horizontal principal point (optical centre). Default = W/2.
cy (px)	540	Vertical principal point. Default = H/2.
Width W (px)	1920	Image width in pixels. Points projected outside $[0, W] \times [0, H]$ are discarded as not visible.
Height H (px)	1080	Image height in pixels.
FOV (auto)	$\approx 53.1^\circ \times 31.0^\circ$	Horizontal and vertical field of view, computed in real time: $FOV_h = 2 \cdot \arctan(W/2fx)$, $FOV_v = 2 \cdot \arctan(H/2fy)$.

Tab — Analysis

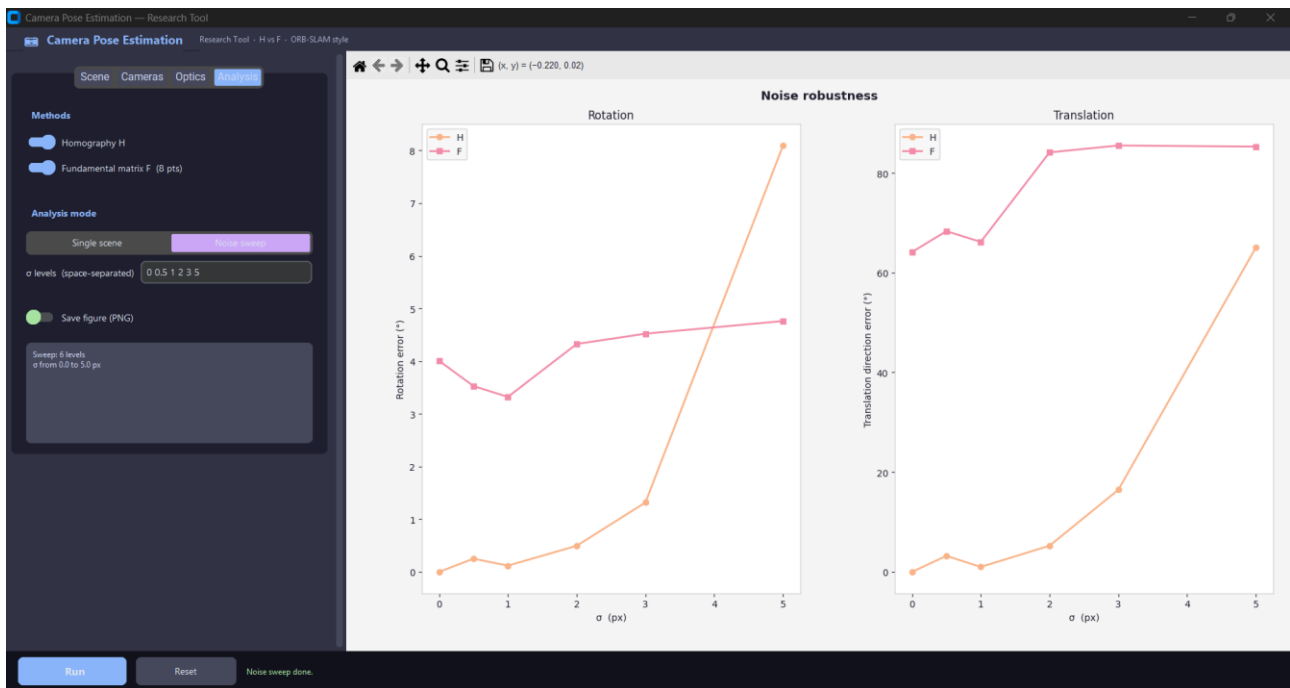


Methods section

- **Homography H switch** — enables or disables the homography estimation branch.
- **Fundamental matrix F (8 pts) switch** — enables or disables the fundamental matrix branch. At least one of the two switches must be ON to launch a Run.

Analysis mode

- **Single scene** — default mode. Runs the full pipeline once with the current parameters. Displays the 3D scene view, reprojection overlays, and the results card.
- **Noise sweep** — repeats the analysis over multiple noise levels σ defined in the " **σ levels**" field (default: 0 0.5 1 2 3 5, space-separated). Produces two error curves — rotation error and translation error — as a function of σ , for both H and F. Useful for comparing robustness against noise.



Save figure switch

When enabled, the matplotlib figure is automatically exported to `pipeline/exports/` after each Run. The filename includes a timestamp (e.g. `result_single_20260507_174532.png`).

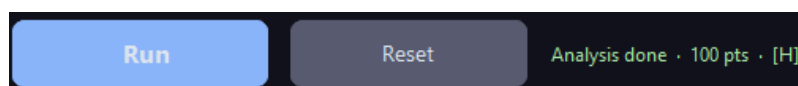
Results card

After each Run in Single scene mode, a summary box at the bottom of the Analysis tab displays:

```
Visible points: 115 H → Err R = 0.703° | Err t = 7.859° F → Err R = 0.000° | Err t = 0.000° Ratio = 0.472 → [H]
```

- **Visible points** — number of 3D points simultaneously visible to both cameras (i.e. projecting within the image bounds of both).
- **Err R** — angular rotation error in degrees. 0° = perfect recovery.
- **Err t** — angular direction error of the translation vector in degrees. Note: the scale of t is not recoverable in monocular estimation, so only the direction is evaluated.
- **Ratio** = $S_H / (S_H + S_F)$. Decision threshold: if Ratio > 0.45, the homography model is selected (shown as [H]). Otherwise the fundamental matrix is used ([F]).

Action buttons (bottom bar)

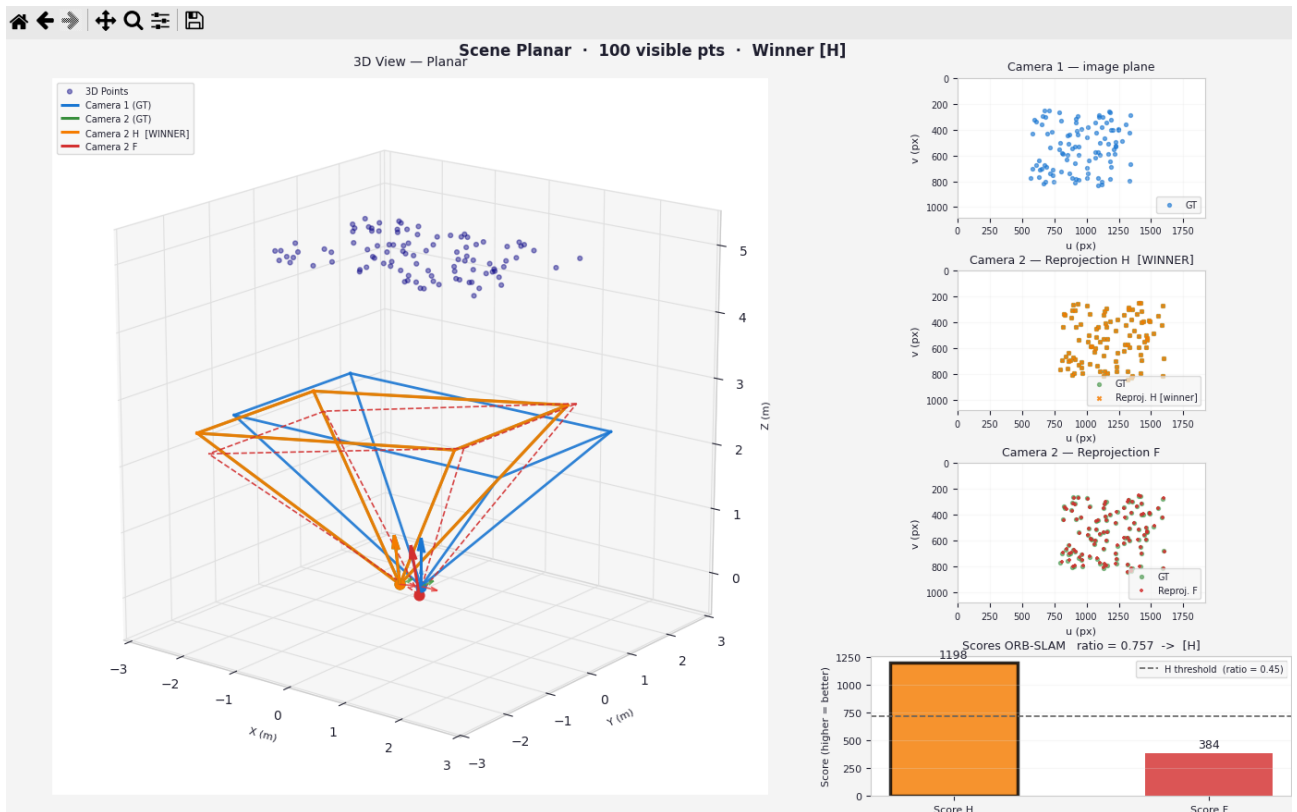


- **Run** — launches the analysis with the current parameters. During computation the button displays "Running..." and is disabled. A progress indicator appears.
- **Reset** — resets all parameters to their default values and clears the display back to the placeholder.

Status bar — displays real-time feedback: "Ready.", "Analysis running...", "Analysis done · 115 pts · [F]", or error messages if something goes wrong.

4. Understanding the Results

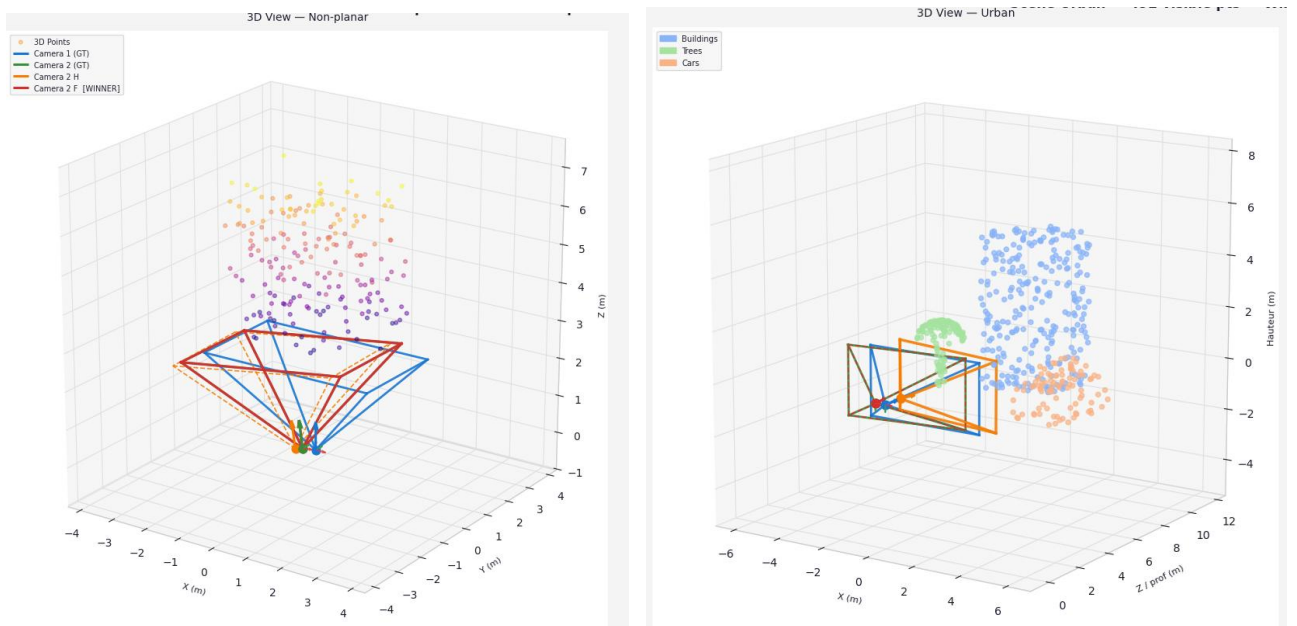
4.1 Layout of the results area



After a Run in Single scene mode, the right-hand canvas is divided into 5 panels:

- 3D scene view
- Camera 1 – image
- Camera 2 – reprojection H
- Camera 2 – reprojection F
- ORB-SLAM score bar

4.2 The 3D scene view



Point cloud colouring

- **Planar / Non-planar** — points are coloured by depth Z using a plasma colormap (purple = near, yellow = far).
- **Urban** — points are coloured by object type:
 - **Blue** = Building
 - **Green** = Tree
 - **Orange** = Car

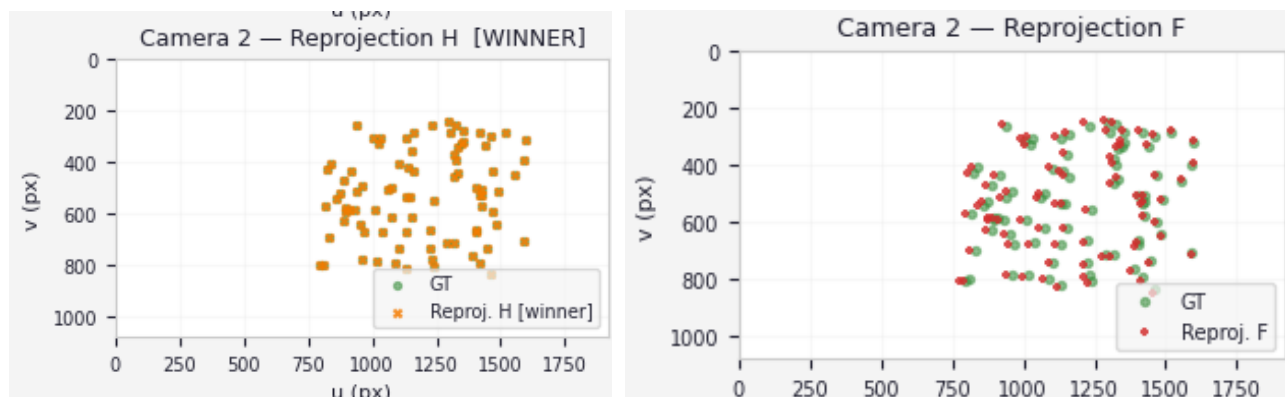
Camera frustums

Each camera is represented by a truncated pyramid (frustum) delimiting its field of view, plus an arrow indicating its viewing direction.

Frustum colour	Camera	Meaning
Blue	Camera 1	Reference camera — fixed ground truth pose.
Green	Camera 2	Ground truth pose of camera 2.
Orange	Camera 2	Pose estimated via H.
Red	Camera 2	Pose estimated via F.

The **winner** (selected model) is drawn with a thick solid line. The **loser** is drawn with a thin dashed line. The closer the estimated frustum (orange or red) overlaps the green frustum, the more accurate the estimation is.

4.3 Reprojection panels: H and F



Principle

Reprojection consists of taking the ground truth 3D points, projecting them into camera 2 using the estimated pose (R, t), and comparing those projected positions with the actual observed 2D positions.

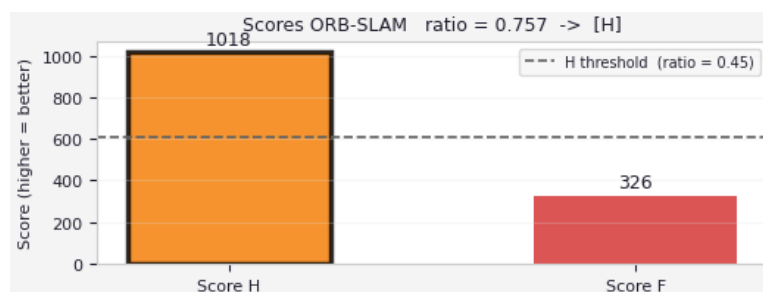
Panel	Points shown	How to read it
Camera 1 — image plane	Blue dots = ground truth projections in camera 1.	Reference view. Shows the spatial distribution of correspondences in the first image.
Camera 2 — Reprojection H	Green dots = ground truth positions. Orange = positions reprojected via H pose.	If it overlaps the green dots exactly, H has correctly estimated the pose. Title shows [WINNER] if H won.

Camera 2 — Reprojection F	Green dots = ground truth positions. Red = positions reprojected via F pose.	If it overlaps the green dots exactly, F has correctly estimated the pose. Title shows [WINNER] if F won.
------------------------------	--	---

Interpreting the overlap quality

Visual result	Interpretation
Perfect overlap	Pose estimation is excellent — error close to 0°.
Uniform offset	Rotation is well estimated but translation has a small error.
Scattered points	Pose is poorly estimated — too much noise, too many outliers, or wrong model selected.

4.4 ORB-SLAM score and ratio



Score H — symmetric transfer error

For each correspondence (px1, px2), the transfer error is measured in both directions:

- **Forward** — project px1 via H into image 2, measure pixel distance to px2.
- **Backward** — project px2 via H^{-1} into image 1, measure pixel distance to px1.

$$S_H = \sum \max(0, 5.99 - d^2_{\text{forward}}) + \max(0, 5.99 - d^2_{\text{backward}})$$

The threshold **5.99** is the 95th percentile of a chi-squared distribution with 2 degrees of freedom (2D transfer error in the image plane).

Score F — Sampson distance

The Sampson distance measures how much each point pair violates the epipolar constraint $x_2^T F x_1 = 0$:

$$d^2_{\text{sampson}} = (x_2^T F x_1)^2 / (\|F x_1\|_{\text{top2}}^2 + \|F^T x_2\|_{\text{top2}}^2) \quad S_F = \sum \max(0, 3.84 - d^2_{\text{sampson}})$$

The threshold **3.84** is the 95th percentile of a chi-squared distribution with 1 degree of freedom (the epipolar constraint is a scalar 1D constraint).

Ratio and decision

$$\text{ratio} = S_H / (S_H + S_F) \quad \text{if ratio} > 0.45 \rightarrow \text{H selected} \quad (\text{scene sufficiently planar}) \quad \text{if ratio} \leq 0.45 \rightarrow \text{F selected}$$

Why 0.45 and not 0.50? F has more degrees of freedom and can fit data well even in planar scenes. The asymmetric threshold deliberately gives H a slight advantage, since H is the correct model for planar scenes and should win even when F is competitive.

4.5 Rotation and translation errors

Rotation error (°)

Measures the minimal angle between the estimated rotation matrix and the ground truth:

```
err_R = arccos( (trace(R_est.T @ R_ref) - 1) / 2 )
```

Principle: any 3D rotation can be written as a rotation by angle θ around an axis (Euler's theorem). The trace of the relative rotation matrix gives this angle directly via $trace = 1 + 2 \cdot \cos(\theta)$.

Value	Interpretation
0.000°	Perfect estimation.
< 1°	Very good estimation.
1° to 5°	Acceptable estimation.
> 5°	Degraded estimation.

Translation error (°)

Why degrees, not metres? The scale of the translation vector is not recoverable without metric ground truth. Only the **direction** of t can be evaluated:

```
u = t_est / ||t_est||      (unit vector of estimated direction)
v = t_ref / ||t_ref||      (unit vector of true direction)
err_t = arccos( |u · v| )
```

The absolute value $|u \cdot v|$ handles the sign ambiguity. $+t$ and $-t$ point in opposite directions but share the same $|\cos|$.

Value	Interpretation
0.000°	Direction perfectly estimated.
< 5°	Very good directional estimation.
5° to 15°	Acceptable estimation.
> 15°	Direction poorly estimated.

5. Code Architecture

5.1 Project structure

```
camera-pose-estimation/
├── pipeline/                    ← Orchestration and GUI
│   ├── core.py
│   ├── gui_tool.py
│   ├── visualize3d.py
│   └── score.py
├── simulation/                 ← Camera parameters + homography
│   ├── camera_model.py
│   ├── projection.py
│   ├── homography.py
│   ├── dlt.py
│   └── scene_urban.py
```

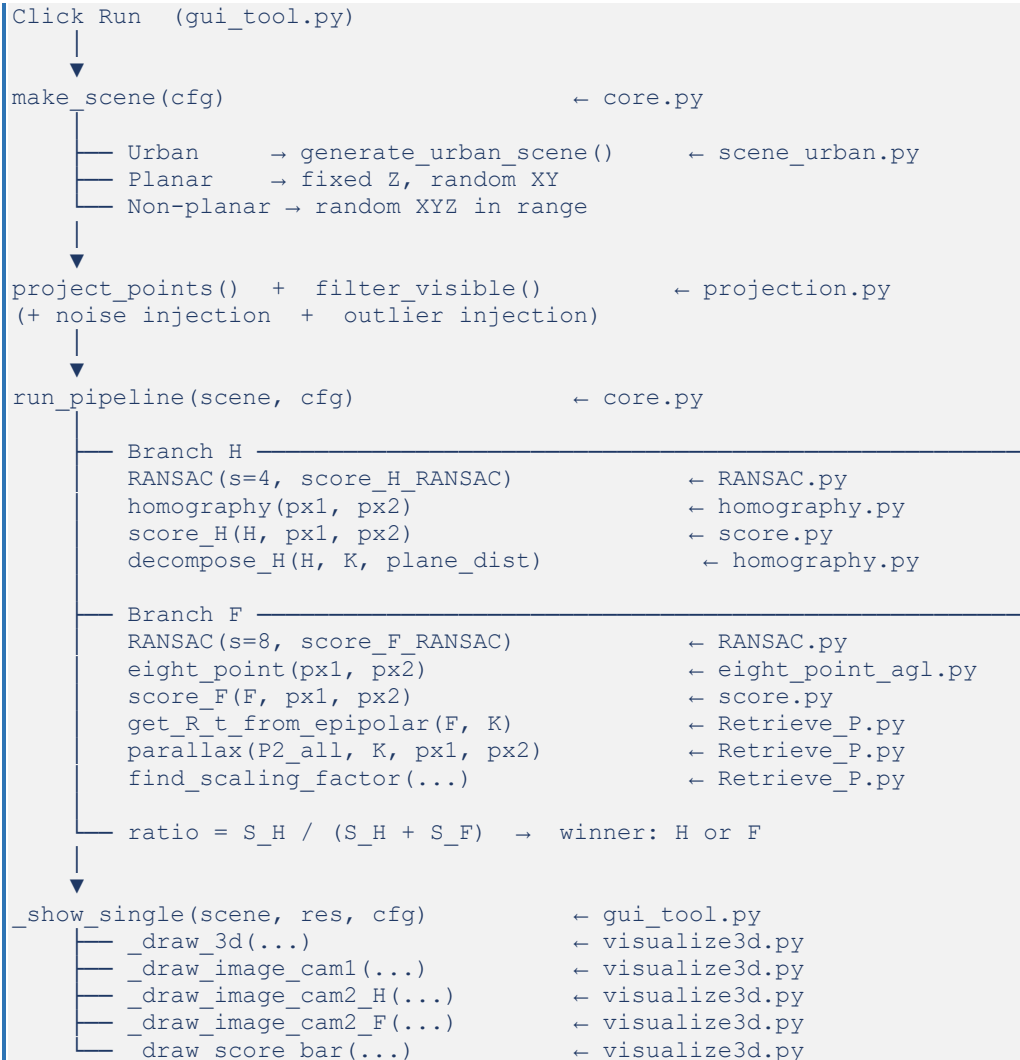
```

└─ eight_points/          ← 8-point
   └─ eight_point_agl.py
   └─ RANSAC.py
   └─ Retrieve_P.py
   └─ triangulation.py

```

5.2 Execution flow

The following sequence is triggered when Run is pressed:



5.3 File-by-file description

pipeline/core.py

Central orchestration file. Contains:

- **Config** — dataclass holding all parameter default values.
- **make_scene(cfg)** — generates the full synthetic scene (3D points → projection → noise → outliers) and returns a dict: {pts3d, px1, px2, K, R_rel, t_rel, R1, t1, urban_colors}.
- **run_pipeline(scene, cfg)** — orchestrates H and F branches, computes scores, makes the model decision, returns errors.
- **run_noise_sweep(cfg)** — loops make_scene + run_pipeline over each σ level to produce the noise sweep data.

pipeline/gui_tool.py

Full GUI built with customtkinter and matplotlib. Contains:

- **App** — main class.
- **Reusable widgets**: EntryRow, SliderRow, TriRow, _SectionTitle.
- **_tab_scene / _tab_cameras / _tab_optics / _tab_analyse** — tab construction functions.
- **_collect_config()** — reads all widgets and builds a Config object.
- **on_run()** — launches the pipeline in a background thread to keep the UI responsive.
- **_show_single() / _show_noise()** — update the matplotlib canvas after a Run.

pipeline/visualize3d.py

All matplotlib drawing functions. Contains:

- **_draw_3d()** — main 3D view: point cloud, frustums, camera axes.
- **_draw_frustum()** — draws one camera frustum (pyramid + directional arrow).
- **_draw_cam_axes()** — draws the three local RGB axes on a camera.
- **_draw_image_cam1/2_H/F()** — reprojection panels in the image plane.
- **_draw_score_bar()** — comparative histogram S_H vs S_F with threshold line.
- **_urban_T()** — coordinate transformation function for the Urban view.

pipeline/score.py

Scoring functions used after RANSAC for the final computation on inliers:

- **score_H(H, px1, px2)** — symmetric transfer error, threshold 5.99.
- **score_F(F, px1, px2)** — Sampson distance, threshold 3.84.

simulation/camera_model.py

- **get_K(fx, fy, cx, cy)** — builds the 3×3 intrinsic matrix K.
- **get_camera_pose(rx, ry, rz, tx, ty, tz)** — builds R and t from Euler angles and translation vector.

simulation/projection.py

- **project_points(pts3d, K, R, t)** — projects 3D world points to 2D pixels via $P = K[R|t]$.
- **filter_visible(px, depth, w, h)** — boolean mask removing points outside the image bounds or behind the camera.

simulation/homography.py

- **homography(pts_src, pts_dst)** — normalised DLT homography estimation (SVD on $2N \times 9$ matrix A), minimum 4 correspondences.
- **decompose_H(H, K, plane_dist, X_ref)** — extracts (R, t) from H. If plane_dist is known (planar scene), returns t in metres; otherwise returns a unit vector. Cheirality test via X_ref.
- **reprojection_error_H()** — RMSE reprojection error for H.

simulation/dlt.py

- **dlt(points_2d, points_3d)** — estimates the 3×4 projection matrix P by normalised Direct Linear Transform, minimum 6 $2D \leftrightarrow 3D$ correspondences.
- **extract_Rt_from_P(P, K)** — extracts R and t from P via SVD decomposition of $K^{-1}P$.

simulation/scene_urban.py

Generates the Urban synthetic scene:

- `_sample_box()` — uniformly samples points on the 6 faces of a box, proportionally to each face area.
- `_sample_tree()` — cylindrical trunk (lateral surface) + flattened hemispherical dome (spherical coordinates, $\phi \in [0, \pi/2]$).
- `_sample_car()` — two stacked boxes (body + roof) via `_sample_box`.
- `generate_urban_scene(rng, n_points)` — assembles the three objects, distributes points (55%/25%/20%), returns (pts3d, colors).

eight_points/eight_point_agl.py

Normalised 8-point algorithm with optimisation:

- `normalize(points)` — isotropic normalisation (centring + scale).
- `linear_eq(points1, points2)` — builds the $N \times 9$ matrix A of the system $Af = 0$.
- `eight_point(pts1, pts2)` — full algorithm: normalise $\rightarrow$ SVD $\rightarrow$ rank-2 enforcement $\rightarrow$ Levenberg–Marquardt optimisation $\rightarrow$ denormalise.
- `min_alg_error(F0, A)` — minimises algebraic error.
- `residual_fun(e, A, state)` — cost functionn.

eight_points/RANSAC.py

- **RANSAC class** — adaptive RANSAC. Iteration count N is updated dynamically as inliers are found: $N = \log(1-p) / \log(1-(1-\epsilon)^s)$, with $p = 0.99$.
- `score_H_RANSAC / score_F_RANSAC` — same logic as `score.py`, used during RANSAC to rank candidate models.

eight_points/Retrieve_P.py

- `get_R_t_from_epipolar(F, K)` — computes $E = K^T F K$, decomposes E by SVD to get t and two rotation candidates $R1, R2$ via the W matrix.
- `P_estimation(t, R1, R2, K)` — assembles the 4 possible projection matrices $K[R|\pm t]$.
- `parallax(P2_all, K, pt1, pt2)` — triangulates points for each solution and selects the one with the most points at positive depth (cheirality test).
- `find_scaling_factor(P2, K, pts1, pts2, pts3D)` — estimates the scale factor by comparing triangulated inter-point distances to ground truth distances.

eight_points/triangulation.py

- `triangulate(P1, P2, pts1, pts2)` — linear triangulation by SVD: for each point pair, builds the 4×4 DLT system and solves it to obtain the homogeneous 3D point.

6. Known Limitations

Current limitations

- **Synthetic data only.** The tool works exclusively with generated scenes. No real image input or actual keypoint matching is implemented.
- **Limited RANSAC control.** The RANSAC toggle enables/disables outlier rejection, but inlier threshold and iteration count are not user-configurable from the GUI.

Potential improvements

- **Real image support** — integrate actual image loading with ORB/SIFT feature extraction and matching to test on real stereo pairs.
- **Advanced RANSAC tuning** — expose inlier threshold and maximum iteration count as GUI sliders.
- **Multi-view extension** — support more than two cameras and incremental structure-from-motion.
- **Export of results** — allow exporting all metrics and estimated matrices to a JSON or CSV file for batch analysis.